



9

User Interface Framework Components

When you are developing a React application, it's typical to rely on an existing **UI library** rather than building one from scratch. There are lots of React UI component libraries available to choose from, and there's no wrong choice, as long as the components make your life simpler.

In this chapter, we delve into the **Material UI** React library, a popular choice for React development. Material UI stands out due to its comprehensive suite of customizable components, adherence to Google's Material Design principles, and extensive documentation, making it an optimal choice for developers seeking efficiency and aesthetic coherence in their UI design. Here are the specific topics that we'll cover:

- Layout and UI organization
- Using **navigation components**
- Collecting user input

- Working with **styles** and **themes**

Technical requirements

You can find the code files present in this chapter on GitHub at

<https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter09>.

You can also find more information about Material UI components and its API at <https://mui.com/material-ui/>.

Layout and organization

Material UI excels in simplifying the complex process of designing application layouts. By offering a robust set of components, specifically **containers** and **grids**, it empowers developers to efficiently structure and organize UI elements.

Containers serve as the foundation, providing a flexible way to encapsulate and align content within the overall layout.

Grids, on the other hand, allow more granular control, enabling precise placement and alignment of components across different screen sizes, ensuring responsiveness and consistency.

This section aims to unpack the functionality of containers and grids within Material UI. We'll explore how these tools can be

leveraged to create intuitive and aesthetically pleasing layouts, which are crucial for enhancing user experience.

Using containers

Aligning components horizontally on a page often presents a significant challenge due to the intricate balance required between spacing, alignment, and responsiveness. This complexity arises from the need to maintain a visually appealing and functional layout across various screen sizes, ensuring that elements are evenly distributed and maintain their intended appearance without unintended overlaps or gaps. The

`Container` component from Material UI is a simple but powerful layout tool. It controls the horizontal width of its children. Let's look at an example to see what's possible:

```
import Typography from "@mui/material/Typography";
import Container from "@mui/material/Container";
export default function MyApp() {
  const textStyle = {
    backgroundColor: "#cfe8fc",
    margin: 1,
    textAlign: "center",
  };
  return (
    <>
      <Container maxWidth="sm">
        <Typography sx={textStyle}>sm</Typography>
      </Container>
    </>
  );
}
```

```
    </Container>
    <Container maxWidth="md">
      <Typography sx={textStyle}>md</Typography>
    </Container>
    <Container maxWidth="lg">
      <Typography sx={textStyle}>lg</Typography>
    </Container>
  </>
);
}
```

This example has three `Container` components, each of which wraps a `Typography` component. The `Typography` component is used to render text in Material UI applications. Each `Container` component used in this example takes a `maxWidth` property. It accepts a breakpoint string value. These breakpoints represent common screen sizes. This example uses small (`sm`), medium (`md`), and large (`lg`). When the screen reaches these breakpoint sizes, the container width will stop growing. Here's what the page looks like when the width is smaller than the `sm` breakpoint:

<code>sm</code>
<code>md</code>
<code>lg</code>

Figure 9.1: The sm breakpoint

Now, if we were to resize the screen so that it was larger than the `md` breakpoint, but smaller than the `lg` breakpoint, here is what it would look like:

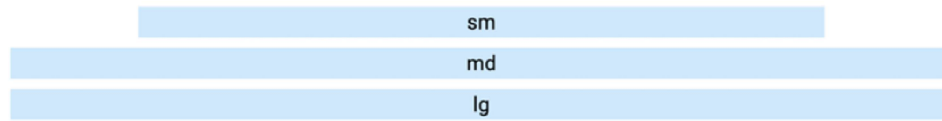


Figure 9.2: The lg breakpoint

Notice how the first container stays at a fixed width now that we've exceeded its `maxWidth` breakpoint. The `md` and `lg` containers just keep growing along with the screen until their breakpoints have been passed.

Let's see what these `Container` components look like when the screen width surpasses all breakpoints:

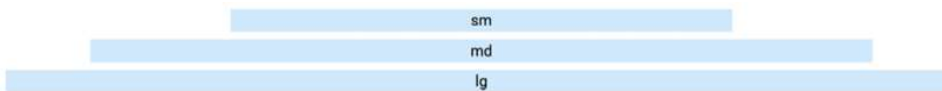


Figure 9.3: All breakpoints

The `Container` component gives you control over how your page elements grow horizontally. They're also responsive, so your layouts will be updated as the screen dimensions change.

In the next section, we'll look at using Material UI components to build more complex and responsive layouts.

Building responsive grid layouts

Material UI has a `Grid` component that we can use to compose responsive complex layouts. At a high level, a `Grid` component can be either a container or an item within a container. By combining these two roles, we can achieve any type of layout for our app. To get familiar with Material UI grid layouts, let's put together an example that uses a common layout pattern that we'll find in many web applications. Here is what the result looks like:



Figure 9.4: A sample responsive grid layout

As you can see, this layout has familiar sections that are typical in many web applications. This is just an example layout; you can use the `Grid` component to build any type of layout you can imagine. Let's look at the code that created this layout:

```
const headerFooterStyle = {
  textAlign: "center",
  height: 50,
};
const mainStyle = {
  textAlign: "center",
  padding: "8px 16px",
};
const Item = styled(Paper)((() => ({
  height: "100%",
  display: "flex",
  alignItems: "center",
  justifyContent: "center",
})));
export default function App() {
  return (
    <Grid container spacing={2} sx={{ backgroundColor: "#F3F6F9" }}>
      <Grid xs={12}>
        <Item sx={headerFooterStyle}>
          <Typography sx={mainStyle}>Header</Typography>
        </Item>
      </Grid>
      <Grid xs="auto">
        <Item>
```

```
    <Stack spacing={1}>
      <Typography sx={mainStyle}>Nav Item 1</Typography>
      <Typography sx={mainStyle}>Nav Item 2</Typography>
      <Typography sx={mainStyle}>Nav Item 3</Typography>
      <Typography sx={mainStyle}>Nav Item 4</Typography>
    </Stack>
  </Item>
</Grid>
<Grid xs>
  <Item>
    <Typography sx={mainStyle}>Main content</Typography>
  </Item>
</Grid>
<Grid xs={12}>
  <Item sx={headerFooterStyle}>
    <Typography sx={mainStyle}>Footer</Typography>
  </Item>
</Grid>
</Grid>
);
}
```

Let's break down how the sections in this layout are created.

We'll start with the header section:

```
<Grid xs={12}>
  <Item sx={headerFooterStyle}>
    <Typography sx={mainStyle}>Header</Typography>
```



```
    </Item>  
  </Grid>
```

The `xs` breakpoint property value of `12` means that the header will always span the entire width of the screen since `12` is the highest value you can use here. Next, let's look at the navigation items:

```
<Grid xs="auto">  
  <Item>  
    <Stack spacing={1}>  
      <Typography sx={mainStyle}>Nav Item 1</Typography>  
      <Typography sx={mainStyle}>Nav Item 2</Typography>  
      <Typography sx={mainStyle}>Nav Item 3</Typography>  
      <Typography sx={mainStyle}>Nav Item 4</Typography>  
    </Stack>  
  </Item>  
</Grid>
```

In the navigation section, we have a grid with the `xs="auto"` prop. It matches the column's size with the width of its content. Also, you can see that we use a `Stack` component to place components in a vertical direction with spacing.

Next, we'll look at the main content section:

```
<Grid xs>
  <Item>
    <Typography sx={mainStyle}>Main content</Typography>
  </Item>
</Grid>
```

The `xs` breakpoint is a true value used to fill all free space after the navigation section in the grid.

In this section, you were introduced to what Material UI has to offer in the way of layouts. You can use the `Container` component to control the width of sections and how they change in response to screen dimension changes. You then learned that the `Grid` component is used to put together more complex grid layouts.

In the following section, we'll look at some of the navigational components found in Material UI.

Using navigation components

Once we have an idea of how the layout of our application is going to look and work, we can start to think about the navigation. This is an important piece of our UI because it's how the user gets around the application, and it will be used frequently. In this section, we'll learn about two of the navigational components offered by Material UI.

Navigating with drawers

The `Drawer` component, just like a physical drawer, slides open to reveal content that is easily accessed. When we're finished, the drawer closes again. This works well for navigation because it stays out of the way, allowing more space on the screen for the active task that the user is engaged with. Let's look at an example, starting with the `App` component:

```
<BrowserRouter>
  <Button onClick={toggleDrawer}>Open Nav</Button>
  <section>
    <Routes>
      <Route path="/first" element={<First />} />
      <Route path="/second" element={<Second />} />
      <Route path="/third" element={<Third />} />
    </Routes>
  </section>
  <Drawer open={open} onClose={toggleDrawer}>
    <div
      style={{ width: 250 }}
      role="presentation"
      onClick={toggleDrawer}
      onKeyDown={toggleDrawer}
    >
      <List component="nav">
        {links.map((link) => (
          <NavLink
            key={link.url}
```

```
      to={link.url}
      style={{ color: "black", textDecoration: "none" }}
    >
      ({({ isActive }) => (
        <ListItemButton selected={isActive}>
          <ListItemText primary={link.name} />
        </ListItemButton>
      )})
    </NavLink>
  )})
</List>
</div>
</Drawer>
</BrowserRouter>
```

Let's look at what's happening here. Everything that this component renders is within the `BrowserRouter` component because the items in the drawer are links to routes:

```
<Button onClick={toggleDrawer}>Open Nav</Button>
<section>
  <Routes>
    <Route path="/first" element={<First />} />
    <Route path="/second" element={<Second />} />
    <Route path="/third" element={<Third />} />
  </Routes>
</section>
```

The `First`, `Second`, and `Third` components are used to render the main application content when the user clicks on a link in the drawer. The drawer itself is opened when the `Open Nav` button is clicked. Let's take a closer look at the state that's used to control this:

```
const [open, setOpen] = useState(false);
const toggleDrawer = ({ type, key }: { type?: string; key?: string }) => {
  if (type === "keydown" && (key === "Tab" || key === "Shift")) {
    return;
  }
  setOpen(!open);
};
```

The `open` state controls the visibility of the drawer. The `onClose` property of the `Drawer` component calls this function, too, meaning that the drawer closes when any of the links within it are activated. Next, let's look at how the links within the drawer are generated:

```
<List component="nav">
  {links.map((link) => (
    <NavLink
      key={link.url}
      to={link.url}
      style={{ color: "black", textDecoration: "none" }}
    >
```

```
    ({ { isActive }) => (  
      <ListItemButton selected={isActive}>  
        <ListItemText primary={link.name} />  
      </ListItemButton>  
    )}  
  </NavLink>  
))}  
</List>
```

The items that are displayed in a `Drawer` component are actually list items, as you can see here. The `links` property has all the link objects with the `url` and `name` properties. Each item in the `items` array is mapped to the `NavLink`, which is used to handle navigation and highlight the active route. Within `NavLink`, we have the `ListItemButton` component, which generates the list item with text by rendering a `ListItemText` component.

Finally, let's look at the default value for the `links` property:

```
const links = [  
  { url: "/first", name: "First Page" },  
  { url: "/second", name: "Second Page" },  
  { url: "/third", name: "Third Page" },  
];
```

Here's what the drawer looks like when it's opened after the screen first loads:

First Page

Second Page

Third Page



Figure 9.5: A drawer showing links to our pages

Try clicking on the `First Page` link. The drawer closes and renders the content of the `/first` route. Then, when you open the drawer again, you'll notice that the `First Page` link is rendered as the active link:

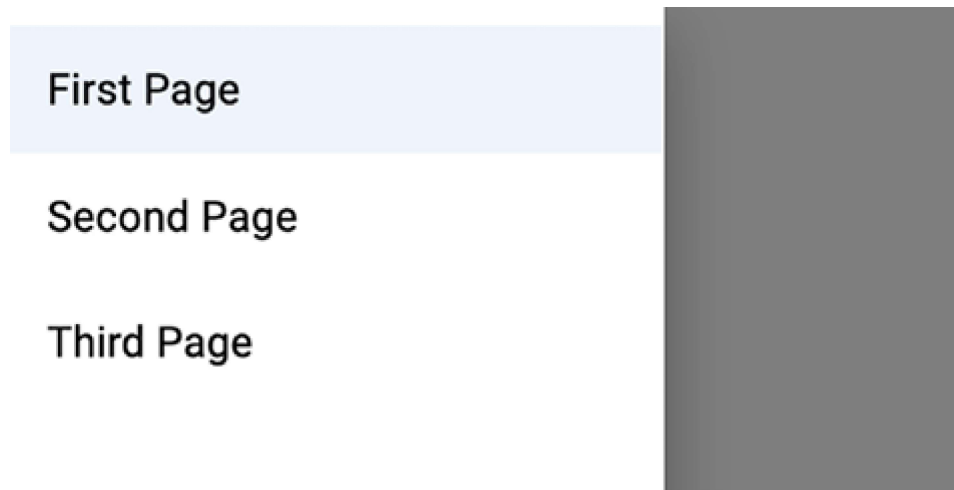


Figure 9.6: The First Page link is styled as the active link in the drawer

In this section, you learned how to use the `Drawer` component as the main navigation for your application. In the following section, we'll look at the `Tabs` component.

Navigating with tabs

Tabs are another common navigation pattern found in modern web apps. The Material UI `Tabs` component lets us use tabs as links and hook them up to a router. Let's look at an example of how to do this. Here is the `App` component:

```
export default function App() {  
  return <RouterProvider router={router} />;  
}
```



```
}  
const router = createBrowserRouter([  
  {  
    path: "/",  
    element: <RouteLayout />,  
    children: [  
      {  
        path: "/page1",  
        element: <Typography>Item One</Typography>,  
      }, // same routes for /page2 and /page3  
    ],  
  },  
]);  
function RouteLayout() {  
  const routeMatch = useRouteMatch(["/", "/page1", "/page2", "/page3"]);  
  const currentTab = routeMatch?.pattern?.path;  
  return (  
    <Box>  
      <Tabs value={currentTab}>  
        <Tab label="Item One" component={Link} to="/page1" value="/page1" />  
        <Tab label="Item Two" component={Link} to="/page2" value="/page2" />  
        <Tab label="Item Three" component={Link} to="/page3" value="/page3" />  
      </Tabs>  
      <Outlet />  
    </Box>  
  );  
}
```

In the interest of space, I've left out the route configuration for `/page2` and `/page3`; it follows the same pattern as `/page1`.

The `Tabs` and `Tab` components from Material UI don't actually render any content underneath the selected tab. It's up to us to provide the content as the `Tabs` component only looks after showing the tabs and marking one of them as selected. This example aims to have the `Tab` components use `Link` components that link to content rendered by routes.

Let's now take a closer look at the `RouteLayout` component. Each `Tab` component uses the `Link` component so that, when it is clicked, the router is activated with the route specified in the `to` property. The `Outlet` component is then used as a child of our route content. To match the active tab, we use a simple approach to handle the current route using `useRouteMatch`:

```
function useRouteMatch(patterns: readonly string[]) {  
  const { pathname } = useLocation();  
  for (let i = 0; i < patterns.length; i += 1) {  
    const pattern = patterns[i];  
    const possibleMatch = matchPath(pattern, pathname);  
    if (possibleMatch !== null) {  
      return possibleMatch;  
    }  
  }  
  return null;  
}
```

The `useRouteMatch` hook uses `useLocation` to get the current `pathname` and then check whether it matches our patterns.

Here's what the page looks like when it first loads:



Figure 9.7: Tabs with the first item active

If you click on the **ITEM TWO** tab, the URL will update, the active tab will change, and the page content below the tabs will change:

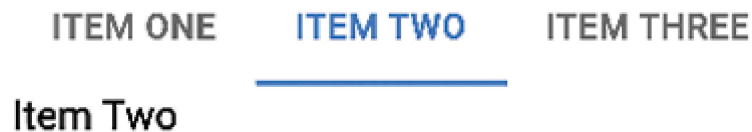


Figure 9.8: Tabs with the second item active

By now, you have learned about two of the navigation approaches that you can use in your Material UI application. The first is to use a `Drawer` that is only displayed when the user needs to access navigational links. The second is to use `Tabs`

that are always visible. In the following section, you'll learn about collecting input from users.

Collecting user input

Collecting input from users can be difficult. There are many nuanced things about every field that we need to consider if we plan on getting the user experience right. Thankfully, the `Form` components available in Material UI take care of a lot of usability concerns for us. In this section, you'll get a brief sampling of the input controls that you can use.

Checkboxes and radio buttons

Checkboxes are useful for collecting `true/false` answers from users, while radio buttons are useful for getting the user to select an option from a short number of choices. Let's take a look at an example of these components in Material UI:

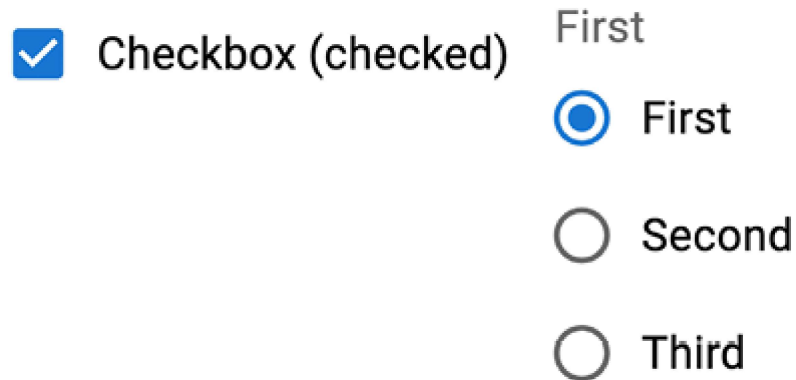
```
export default function Checkboxes() {  
  const [checkbox, setCheckbox] = React.useState(false);  
  const [radio, setRadio] = React.useState("First");  
  return (  
    <div>  
      <FormControlLabel  
        label={'Checkbox ${checkbox ? "(checked)" : ""}'}  
        control={  
          <Checkbox
```

```
        checked={checkbox}
        onChange={() => setCheckbox(!checkbox)}
      />
    }
  />
  <FormControl component="fieldset">
    <FormLabel component="legend">{radio}</FormLabel>
    <RadioGroup value={radio} onChange={(e) => setRadio(e.target.value)}>
      <FormControllabel value="First" label="First" control={<Radio />} />
      <FormControllabel value="Second" label="Second" control={<Radio />} />
      <FormControllabel value="Third" label="Third" control={<Radio />} />
    </RadioGroup>
  </FormControl>
</div>
);
}
```

This example has two pieces of state information. The `checkbox` state controls the value of the `Checkbox` component, while the `radio` value controls the state of the `RadioGroup` component. The `checkbox` state is passed to the `checked` property of the `Checkbox` component, while the `radio` state is passed to the `value` property of the `RadioGroup` component. Both components have `onChange` handlers that call their respective state setter functions: `setCheckbox()` and `setRadio()`. You'll notice that many other Material UI components are involved in the display of these controls. For example, the label for the `checkbox` is displayed using the

`FormControlLabel` component, while the radio control uses a `FormControl` component and a `FormLabel` component.

Here is what the two input controls look like:



☒ **Checkbox (checked)**

☒ **First**

☐ **Second**

☐ **Third**

Figure 9.9: A checkbox and a radio group

The labels for both of these controls are updated to reflect the state of the components as they change. The checkbox labels show whether the checkbox is checked, and the radio labels show the currently selected value. In the next section, we'll look at text inputs and select components.

Text inputs and select inputs

Text fields allow our users to enter text, while `Select` allows them to choose from several options. The difference between

selects and radio buttons is that selects require less space on the screen since the options are only displayed when the user opens the options menu.

Let's look at a `Select` component now:

```
import { useState } from "react";
import InputLabel from "@mui/material/InputLabel";
import MenuItem from "@mui/material/MenuItem";
import FormControl from "@mui/material/FormControl";
import Select from "@mui/material/Select";
export default function MySelect() {
  const [value, setValue] = useState<string | undefined>();
  return (
    <FormControl>
      <InputLabel id="select-label">My Select</InputLabel>
      <Select
        labelId="select-label"
        id="select"
        label="My Select"
        value={value}
        onChange={(e) => setValue(e.target.value)}
        inputProps={{ id: "my-select" }}
      >
        <MenuItem value="first">First</MenuItem>
        <MenuItem value="second">Second</MenuItem>
        <MenuItem value="third">Third</MenuItem>
      </Select>
    </FormControl>
  );
}
```

```
);  
}
```

The value state used in this example controls the selected value in the `Select` component. When the user changes their selection, the `set_value()` function changes the value.

The `MenuItem` component is used to specify the available options in the `select` field; the `value` property is set as the value state when a given item is selected. Here's what the `select` field looks like when the menu is displayed:

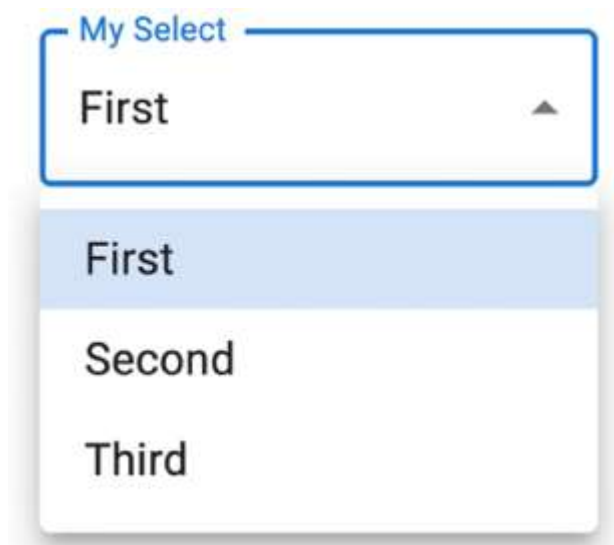


Figure 9.10: A menu with the first item active

Next, let's look at a `TextField` component example:


```
export default function MyTextInput() {  
  const [value, setValue] = useState("");  
  return (  
    <TextField  
      label="Name"  
      value={value}  
      onChange={(e) => setValue(e.target.value)}  
      margin="normal"  
    />  
  );  
}
```

The value state controls the value of the text input and changes as the user types. Here's what the `text` field looks like:



Figure 9.11: A text field with user-provided text

Unlike other `FormControl` components, the `TextField` component doesn't require several other supporting components. Everything that we need can be specified via properties. In the next section, we'll look at the `Button` component.

Working with buttons

Material UI buttons are very similar to HTML button elements. The difference is that they're React components that work well with other aspects of Material UI, such as theming and layout. Let's look at an example that renders different styles of buttons:

```
type ButtonColor = "primary" | "secondary";
export default function App() {
  const [color, setColor] = useState<ButtonColor>("secondary");
  const updateColor = () => {
    setColor(color === "secondary" ? "primary" : "secondary");
  };
  return (
    <Stack direction="row" spacing={2}>
      <Button variant="contained" color={color} onClick={updateColor}>
        Contained
      </Button>
      <Button color={color} onClick={updateColor}>
        Text
      </Button>
      <Button variant="outlined" color={color} onClick={updateColor}>
        Outlined
      </Button>
      <IconButton color={color} onClick={updateColor}>
        <AndroidIcon />
      </IconButton>
    </Stack>
  );
}
```

```
);  
}
```

This example renders four different button styles. We're using the `Stack` component to render the row of buttons. When the buttons are clicked on, the state toggles to primary and vice versa.

Here's what the buttons look like when they're first rendered:



Figure 9.12: Four styles of Material UI buttons

Here's what the buttons look like when they've each been clicked on:



Figure 9.13: What the buttons look like after they've been clicked on

In this section, you learned about some of the user input controls available in Material UI. `Checkboxes` and `radio` buttons are useful when the user needs to turn something on or off or choose an option. Text inputs are necessary when the user needs to type in some text, while `select` fields are useful when you have a list of options to choose from but limited space to display those options. Finally, you learned that Material UI has several styles of buttons that can be used when the user needs to initiate an action. In the following section, we'll look at how styles and themes work in Material UI.

Working with styles and themes

Included with Material UI are systems for extending the styles of UI components and extending **theme styles** that are applied to all components. In this section, you'll learn about using both of these systems.

Making styles

Material UI comes with a `styled()` function that can be used to create styled components based on JavaScript objects. The return value of this function is a new component with the new styles applied.

Let's take a closer look at this approach:

```
const StyledButton = styled(Button)(({ theme }) => ({
  "&.MuiButton-root": { margin: theme.spacing(1) },
  "&.MuiButton-contained": { borderRadius: 50 },
  "&.MuiButton-sizeSmall": { fontWeight: theme.typography.fontWeightLight },
}));

export default function App() {
  return (
    <>
      <StyledButton>First</StyledButton>
      <StyledButton variant="contained">Second</StyledButton>
      <StyledButton size="small" variant="outlined">
        Third
      </StyledButton>
    </>
  );
}
```

The names used in this style (`MuiButton-root` , `MuiButton-contained` , and `MuiButton-sizeSmall`) aren't something that we came up with. These are part of the **Button CSS API**. The root style is applied to all buttons, so, in this example, all three buttons will have the margin value that we've applied here. The `contained` style is applied to buttons that use the contained variant. The `sizeSmall` style is applied to buttons that have a small value for the size property.

Here's what the custom button styles look like:



Figure 9.14: Buttons using customized styles

Now that you know how to change the look and feel of individual components, it's time to think about customizing the look and feel of the application as a whole.

Customizing themes

Material UI comes with a default theme. We can use this as the starting point to create our own theme. There are two main steps to creating a new theme in Material UI:

1. Use the `createTheme()` function to customize the default theme settings and return a new theme object.
2. Use the `ThemeProvider` component to wrap our application so that the appropriate theme is applied.

Let's look at how this process works in practice:

```
import Menu from "@mui/material/Menu";
import MenuItem from "@mui/material/MenuItem";
import { ThemeProvider, createTheme } from "@mui/material/styles";
```

```
const theme = createTheme({
  typography: {
    fontSize: 11,
  },
  components: {
    MuiMenuItem: {
      styleOverrides: {
        root: {
          marginLeft: 15,
          marginRight: 15,
        },
      },
    },
  },
});
export default function App() {
  return (
    <ThemeProvider theme={theme}>
      <Menu anchorEl={document.body} open={true}>
        <MenuItem>First Item</MenuItem>
        <MenuItem>Second Item</MenuItem>
        <MenuItem>Third Item</MenuItem>
      </Menu>
    </ThemeProvider>
  );
}
```

The custom theme that we've created here does two things:

- It changes the default font size for all components to **11**.

- It updates the left and right margin values for the `MenuItem` components.

Many values can be set in a Material UI theme; refer to the customization documentation for more. The `components` section is used for component-specific customizations. This is useful when you need to style every instance of a component in your application.

Summary

This chapter was a very brief introduction to Material UI, the most popular React UI framework. We started by looking at the components used to assist with the layout of our pages. We then looked at components that can help the user navigate around your application. Next, you learned how to collect user input using Material UI form components. Finally, you learned how to style your Material UI using styles and modifying themes.

The insights gained from this chapter allow you to build complex interfaces without the overhead of developing UI components from scratch, accelerating your development process. Furthermore, React application development inherently relies on the synergistic use of various auxiliary libraries. A deep understanding of the React ecosystem and its key libraries em-

powers developers to rapidly prototype and iterate on their applications, making development effective.

In the next chapter, we'll look at ways to improve the efficiency of your component state updates using the latest functionality available in the latest version of React.